

Algorytm Gaussa Newtona

Dawid Skowroński

2026-05-06

```
library(ggplot2)
library(MASS)

# Parametry
n <- 10
x <- (1:10)/2
b1 <- 10
b2 <- 3
b3 <- 2
beta <- c(b1,b2,b3)
sig <- 1/2

set.seed(2137)
y_gen <- (b1*x)/(b2+x)+b3 + rnorm(10,mean = 0, sd = sig)

fun_g <- function(x,beta){
  return((beta[1] * x) / (beta[2] + x) + beta[3])
}

grad_g <- function(x, beta){
  return(c(x/(beta[2]+x), - (beta[1] *x)/(beta[2]+x)^2,1))
}

grad_g(1/2, beta= beta)

## [1] 0.1428571 -0.4081633 1.0000000

# Wzór na g(x,beta)

# Definiujemy macierz G
mac_G <- function(wek_x, beta){
  n <- length(wek_x)
  p <- length(beta)
  mac <- matrix(nrow = n, ncol = p)
  for (i in 1:n){
    mac[i, ] <- grad_g(wek_x[i], beta = beta)
  }
  return(mac)
}
```

```

# Algorytm Gaussa Newtona
G_N <- function(x,y, beta_0 = c(8,2,1), eps=0.0001 , M=100){

  n <- length(x)
  p <- length(beta_0)

  stare_beta <- beta_0

  y_prog0 <- fun_g(x, beta_0)
  SSR <- sum((y-y_prog0)^2)

  # Krok zerowy
  historia <- data.frame()

  for (i in 1:M){

    y_prog <- fun_g(x, stare_beta)
    G <- mac_G(x, beta = stare_beta)

    SSR <- sum((y-y_prog)^2)
    krok <- ginv(t(G) %*% G) %*% t(G) %*% (y-y_prog)

    nowe_beta <- stare_beta + krok

    historia <- rbind(
      historia,
      data.frame(
        krok = i-1,
        beta1 = stare_beta[1],
        beta2 = stare_beta[2],
        beta3 = stare_beta[3],
        sigma_kw = SSR / (n - p)
      )
    )

    # Warunek stopu
    if (sqrt(sum((nowe_beta-stare_beta)^2))<eps){
      break
    }
    stare_beta <- nowe_beta
  }

  y_prog <- fun_g(x, nowe_beta)
  G <- mac_G(x, beta = nowe_beta)

  SSR <- sum((y-y_prog)^2)

  historia <- rbind(
    historia,
    data.frame(
      krok = i,
      beta1 = nowe_beta[1],
      beta2 = nowe_beta[2],

```

```

        beta3 = nowe_beta[3],
        sigma_kw = SSR / (n - p)
    )
}

return(return(historia))
}

G_N(x,y_gen)

```

```

##   krok    beta1    beta2    beta3  sigma_kw
## 1    0  8.00000  2.000000  1.000000  2.7625563
## 2    1 11.71695  3.381448  1.452789  0.3183481
## 3    2 12.19515  3.151323  1.223231  0.2738705
## 4    3 12.22165  3.187911  1.228887  0.2735583
## 5    4 12.21970  3.185565  1.227588  0.2735582
## 6    5 12.21985  3.185733  1.227666  0.2735582
## 7    6 12.21984  3.185721  1.227661  0.2735582

```